

Практическое занятие №2

Операторы. Преобразование типов

Цель работы: изучить арифметические, логические операторы и операторы сравнения. Научиться работать с преобразованием типов. Реализовать простую логику для проекта.

Время: 2 часа.

Лабораторное оборудование: персональный компьютер с доступом в Интернет.

Краткие теоретические сведения

Арифметические операции

Арифметические операторы используются для выполнения математических действий над числами: «+» сложение; «-» вычитание; «*» умножение; «/» деление; «%» остаток от деления; «**» возведение в степень.

Например:

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
console.log(a ** b); // 1000
```

Арифметические операторы часто используются для расчёта возраста, количества человек, стоимости, скидки, времени достижения цели и других показателей.

Инкремент и декремент

Для увеличения или уменьшения значения переменной на единицу используются операторы «++» и «--». Различают префиксную и постфиксную формы написания:

- «a++» – постфиксная форма: возвращает старое значение, затем увеличивает переменную.
- «++a» – префиксная форма: сначала увеличивает переменную, затем возвращает новое значение.
- Аналогично работают «a--» и «--a».

Например:

```
let a = 5;

console.log(a++); // 5 – сначала возвращается старое значение, потом a
увеличивается
console.log(a);   // 6

let b = 5;
console.log(++b); // 6 – сначала b увеличивается, потом возвращается новое
значение
console.log(b);   // 6
```

В реальных проектах *инкремент* и *декремент* применяются для счётчиков, нумерации, пагинации, циклов и изменения состояния.

Операторы сравнения

Операторы сравнения возвращают логическое значение *true* или *false*:

- «>» – больше;
- «<» – меньше;
- «>=» – больше или равно;
- «<=» – меньше или равно;
- «==» – нестрогое равенство;
- «===» – строгое равенство;
- «!=» – нестрогое неравенство;
- «!==» – строгое неравенство.

Например:

```
console.log(10 > 5);    // true
console.log(10 < 5);    // false
console.log(10 >= 10);  // true
console.log(10 <= 9);   // false
```

Важно различать «==» и «===». *Например:*

```
console.log(5 == '5'); // true – происходит приведение типов
console.log(5 === '5'); // false – сравниваются и значение, и тип
console.log(null == undefined); // true
console.log(null === undefined); // false
```

Оператор «==» выполняет неявное приведение типов, поэтому его поведение может быть неочевидным. Оператор «===» сравнивает значения без приведения типов и считается более предпочтительным в большинстве ситуаций. В проектной логике лучше использовать «===» и «!==», чтобы избежать неожиданных результатов.

Логические операторы

Логические операторы используются для объединения нескольких условий:

- «**&&**» – логическое «**И**». Возвращает **true**, если оба операнда истинны;
- «**||**» – логическое «**ИЛИ**». Возвращает **true**, если хотя бы один операнд истинен;
- «**!**» – логическое «**НЕ**». Инвертирует логическое значение.

Например:

```
let isAdult = true;
let isWithinLimit = true;

console.log(isAdult && isWithinLimit); // true
console.log(isAdult || isWithinLimit); // true
console.log(!isAdult);                // false
```

Логические операторы часто используются для проверки условий регистрации, бронирования, доступа или валидации данных.

Логическое короткое замыкание

Операторы «**&&**» и «**||**» работают по принципу короткого замыкания.

- В выражении **A && B** JavaScript сначала вычисляет **A**. Если **A** ложно, то **B** уже не вычисляется, потому что всё выражение не может быть истинным.
- В выражении **A || B** JavaScript сначала вычисляет **A**. Если **A** истинно, то **B** не вычисляется, потому что всё выражение уже истинно.

Например:

```
let user = null;

if (user && user.age >= 18) {
  console.log("Пользователь совершеннолетний");
} else {
  console.log("Пользователь не найден или несовершеннолетний");
}
```

Здесь обращение «**user.age**» не вызовет ошибку, если «**user**» равен «**null**», потому что благодаря короткому замыканию правая часть не будет вычисляться.

В проекте это можно использовать так:

```
let canRegister = isAdult && isWithinLimit && !isBlacklisted;
```

Преобразование типов

JavaScript поддерживает явное и неявное преобразование типов. Явное преобразование выполняется с помощью функций:

- *Number()* – преобразование к числу;
- *String()* – преобразование к строке;
- *Boolean()* – преобразование к логическому типу;
- *parseInt()* – преобразование строки в целое число;
- *parseFloat()* – преобразование строки в число с плавающей точкой.

Например:

```
let value = "123";

console.log(Number(value)); // 123
console.log(String(456));  // "456"
console.log(Boolean(1));   // true
console.log(Boolean(0));   // false
```

Особенности преобразования:

```
Number("123") // 123
Number("12abc") // NaN
Number("") // 0
Number(null) // 0
Number(undefined) // NaN

String(123) // "123"
Boolean("0") // true
Boolean("") // false
```

К ложным значениям относятся: *false*, *0*, *-0*, *0n*, *""*, *null*, *undefined*, *NaN*. Все остальные значения при приведении к логическому типу становятся *true*.

Получение данных от пользователя через prompt()

Функция *prompt()* всегда возвращает «строку» или «*null*», если пользователь нажал «Отмена».

Например:

```
let birthYear = prompt("Введите год рождения:");
console.log(typeof birthYear); // string
```

Если сразу использовать полученное значение в арифметике, может возникнуть неявное преобразование. *Например, при сложении строка может склеиться с числом:*

```
let guests = prompt("Количество гостей:"); // "2"
let total = 1 + guests;
console.log(total); // "12", а не 3
```

Чтобы избежать подобных ошибок, данные от пользователя нужно явно преобразовывать:

```
let guests = Number(prompt("Количество гостей:"));
let totalPeople = 1 + guests;
console.log(totalPeople); // 3
```

Работа с датой

Для получения текущего года используется объект **Date**:

```
const currentYear = new Date().getFullYear();
console.log(currentYear); // например, 2026
```

Задания на выполнение практической работы

Этап 1: Базовый блок

Сценарий: вы разрабатываете модуль валидации и обработки данных для формы регистрации на мероприятие.

Задание 1.1 Работа с арифметическими операторами и преобразованием типов

1. Создайте файл «**base_task_2.js**».
2. Запросите у пользователя год его рождения с помощью **prompt()**.
3. Вычислите возраст пользователя, используя текущий год (получите его с помощью **new Date().getFullYear()**).

4. Запросите у пользователя количество гостей, которых он планирует взять с собой.

5. Рассчитайте общее количество человек (пользователь + гости).

6. Выведите в консоль возраст пользователя и общее количество человек.

Задание 1.2 Работа с операторами сравнения и логическими операторами

7. Проверьте, является ли пользователь совершеннолетним (возраст ≥ 18).
8. Проверьте, не превышает ли общее количество человек максимально допустимое значение (например, 5 человек).

9. Используя логические операторы, определите, можно ли пользователю зарегистрироваться: регистрация возможна, если пользователь совершеннолетний **И** общее количество человек не превышает лимит.

10. Выведите в консоль результаты проверок и итоговое решение («Регистрация разрешена» или «Регистрация запрещена»).

Задание 1.3 Глубокая работа с преобразованием типов и шаблонными строками

11. Запросите у пользователя его имя и любимое число.
12. Преобразуйте любимое число в строку и сохраните в переменную.
13. Преобразуйте имя пользователя в верхний регистр.
14. Используя шаблонные строки, выведите сообщение:

Пользователь: [Имя].

Любимое число: [Число].

Тип числа: [Тип].

Возраст: [Возраст].

15. Выведите в консоль тип данных каждого полученного значения (имя, возраст, любимое число, преобразованное число).

Пример итогового вывода в консоль:

Возраст пользователя: 26

Общее количество человек: 4

Совершеннолетний: true

Не превышает лимит: true

Регистрация разрешена

Пользователь: Юрий. Любимое число: 7. Тип числа: string. Возраст: 26.

string

number

string

number

Этап 2: Проектный блок (Индивидуальное задание)

Цель: реализовать core-логику вашего проекта с использованием изученных операторов и преобразований.

Задание 2.1 Создание переменных и констант к индивидуальному проекту

1. Создайте файл `«project_logic.js»`.
2. Напишите код, который реализует ключевую функцию вашего проекта, используя изученные операторы и преобразования типов.
3. Обязательные требования:
 - Использование как минимум 3-х разных типов операторов (арифметические, сравнения, логические).

- Явное преобразование типов данных (например, `Number()`, `String()`).
- Использование условных конструкций (*`if-else`* или *`switch-case`*).
- Вывод результатов в консоль с помощью шаблонных строк.

Пример для финансового трекера («Анализ финансового здоровья»):

```
let monthlyIncome = Number(prompt("Введите ваш месячный доход:"));
let monthlyExpenses = Number(prompt("Введите ваши месячные расходы:"));
let savingsGoal = Number(prompt("Введите вашу цель накоплений:"));

let balance = monthlyIncome - monthlyExpenses;
let savingsTime = savingsGoal / balance;

console.log(`Месячный баланс: ${balance} руб.`);
console.log(`Время для достижения цели: ${savingsTime} месяцев`);

if (balance > 0) {
  console.log("У вас положительный финансовый баланс!");
  if (savingsTime <= 12) {
    console.log("Цель будет достигнута менее чем за год!");
  } else {
    console.log("Для достижения цели потребуется более года");
  }
} else {
  console.log("Отрицательный баланс! Пересмотрите расходы.");
}
```

Пример для системы бронирования («Проверка доступности времени»):

```
const WORK_START = 9;
const WORK_END = 18;
const BOOKING_DURATION = 2;

let userTime = Number(prompt("На который час хотите забронировать?"));
let currentBookings = 3;

let isWithinWorkHours = userTime >= WORK_START && userTime < WORK_END;
let isTimeSlotAvailable = currentBookings < 5;
let endTime = userTime + BOOKING_DURATION;

console.log(`Выбранное время: ${userTime}:00`);
console.log(`Конец брони: ${endTime}:00`);
console.log(`В рабочее время: ${isWithinWorkHours}`);
console.log(`Есть свободные слоты: ${isTimeSlotAvailable}`);

if (isWithinWorkHours && isTimeSlotAvailable && endTime <= WORK_END) {
  console.log("Бронь возможна! Время подтверждено.");
} else {
  console.log("Бронь невозможна. Выберите другое время.");
}
```

Пример библиотеки («Проверка сроков возврата книги»):

```
const MAX_LOAN_DAYS = 14;
let loanDate = new Date("2024-03-01"); // Дата выдачи
let currentDate = new Date(); // Текущая дата

let daysDiff = Math.floor((currentDate - loanDate) / (1000 * 60 * 60 * 24));
let daysLeft = MAX_LOAN_DAYS - daysDiff;
let isOverdue = daysDiff > MAX_LOAN_DAYS;

console.log(`Прошло дней: ${daysDiff}`);
console.log(`Осталось дней: ${daysLeft}`);
console.log(`Просрочка: ${isOverdue}`);

if (isOverdue) {
  let fine = (daysDiff - MAX_LOAN_DAYS) * 10; // Штраф 10 руб./день
  console.log(`Просрочено! Штраф: ${fine} руб.`);
} else if (daysLeft <= 3) {
  console.log("Скоро срок сдачи! Верните книгу вовремя.");
} else {
  console.log("Срок сдачи не нарушен.");
}
```

Контрольные вопросы

1. В чем разница между операторами `==` и `===`? Приведите пример ситуации, когда использование `==` может привести к неожиданным результатам в вашем проекте.
2. Почему важно явно преобразовывать данные, полученные от пользователя через `prompt()`? Какие проблемы могут возникнуть при неявном преобразовании типов в арифметических операциях?
3. Объясните разницу между префиксной и постфиксной формами инкремента/декремента (`++a` vs `a++`). Где может потребоваться каждая из них в реальном проекте?
4. Как работает «логическое короткое замыкание» (операторы: `&&`, `||`) в JavaScript? Приведите практический пример использования этого механизма в вашем проекте.
5. Когда уместно использовать тернарный оператор вместо конструкции `if-else`? Какие преимущества и ограничения он имеет?

Содержание отчёта

Отчет выполняется единым, в текстовом (электронном) виде, который содержит номер лабораторной работы, тему и цель работы, а также краткое описание выполняемых операций и скриншоты результатов выполнения, ответы на контрольные вопросы, выводы. Отчёт оформляется согласно ГОСТ 7.32-2017.

Структура отчёта должна содержать следующее:

- наименование и номер работы;
- тему и цель работы;
- раздел «Постановка задачи», где описываются задачи, которые необходимо решить в ходе выполнения практической работы;
- раздел «Ход работы», где описывается ход выполнения работы с кратким описанием выполняемых операций и приведенных результатов выполнения в виде скриншотов и листинга кода;
- ответы на контрольные вопросы (при наличии);
- выводы.